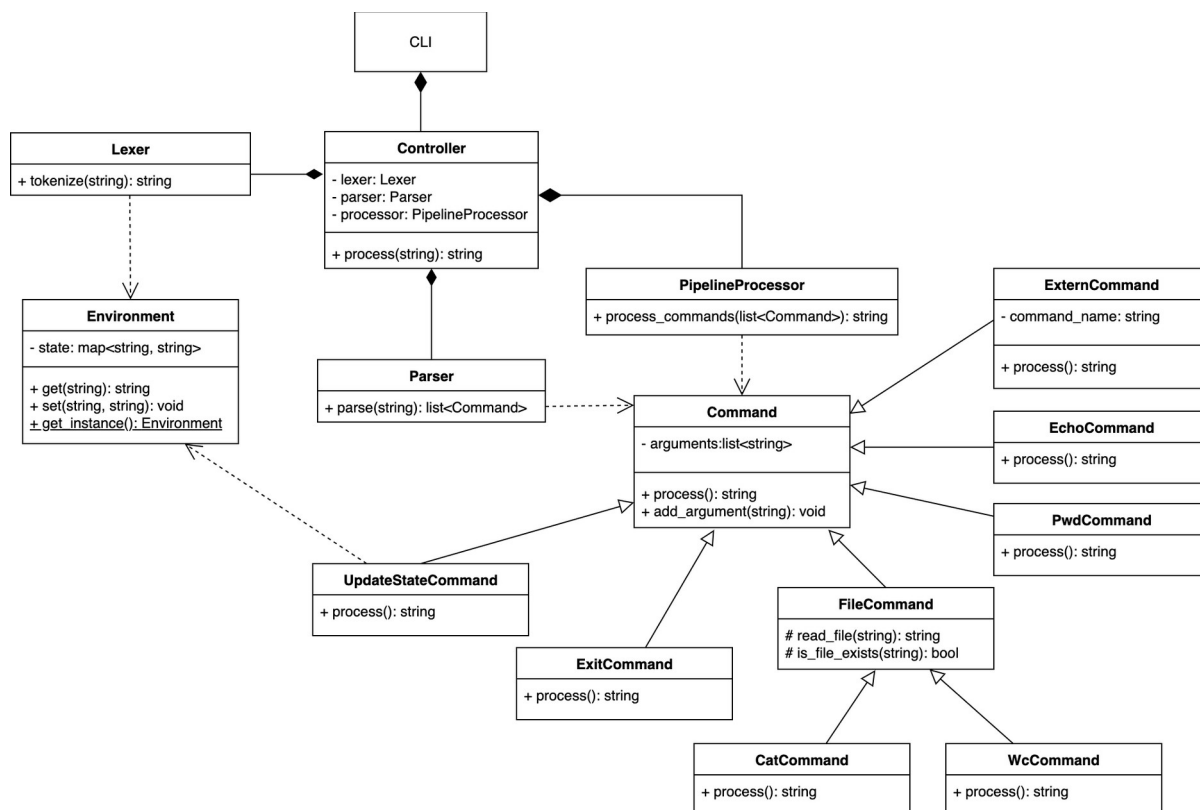




CLI – класс, который отвечает за взаимодействие с пользователем: ввод команд и вывод результата работы системы. Он обращается к контроллеру для обработки пользовательского ввода.

Controller – координирующий компонент, который объединяет работу лексера (**Lexer**), парсера (**Parser**) и обработчика пайплайнов (**PipelineProcessor**). Метод `process` принимает строку ввода, передаёт её на лексический анализ, затем на синтаксический, а результат интерпретируется обработчиком пайплайна.

Lexer – модуль, выполняющий разбиение пользовательского ввода на лексемы. Он связан с объектом **Environment**, так как при разборе может потребоваться доступ к переменным окружения.

Environment – объект-синглтон, хранящий текущее состояние окружения (переменные). Имеет методы для получения и установки значений по ключу. Используется как лексером, так и отдельными командами, которые изменяют состояние (**UpdateStateCommand**).

Parser – выполняет синтаксический анализ строки, возвращая список команд для дальнейшей обработки.

PipelineProcessor – отвечает за выполнение команды или набора команд, соединённых пайпами. Принимает список объектов типа **Command** и обеспечивает последовательное выполнение, передавая результат одной команды на вход следующей.

Command – абстрактный класс, представляющий интерфейс для всех конкретных команд. Каждая команда имеет список аргументов и реализует метод `process`, возвращающий строковый результат.

Классы-наследники **Command** реализуют конкретные команды.

При реализации класса-наследника **GrepCommand** была добавлена поддержка флагов `-i` (игнорирование регистра), `-w` (определение вхождения как слова, а не подстроки) и аргумента `-A` (вывод следующих `N` строк после совпадения, где `N` — значение аргумента) при помощи библиотеки **sxxopts** (<https://github.com/jarro2783/sxxopts>).

Причины выбора:

1. *Header-only*
Библиотека не требует отдельной сборки или установки, её достаточно подключить через один `#include <sxxopts.hpp>`. Это особенно удобно для учебных и кроссплатформенных проектов — можно просто добавить `FetchContent` в `CMakeLists.txt`, и всё соберётся автоматически.
2. *Современный интерфейс (C++17)*
`sxxopts` написана в современном стиле, безопасна по типам и поддерживает работу с `std::string`.
3. *Минимальная зависимость.*
В отличие от `Boost.Program_options`, библиотека не тянет за собой огромный фреймворк и не усложняет сборку.
4. *Поддержка коротких и длинных ключей*
Позволяет легко описывать опции вроде `-i`, `-w`, `-A`, а также позиционные аргументы (например, шаблон поиска и имя файла), что идеально подходит под поведение настоящего `grep`.
5. *Хорошая документация и активная поддержка*
Библиотека используется в реальных CLI-утилитах и активно обновляется.

Рассмотренные альтернативы

1. *Boost.Program_options*
Слишком тяжёлая, требует установки Boost и настройки путей.
2. *CLI11*
Мощная, избыточна для одной команды `grep`.
3. *getopt*
Устаревшие C-интерфейсы, не поддерживают типобезопасность и позиционные аргументы.

ErrorHandler — объект-синглтон, отвечающий за обработку ошибок во всех классах программы (где они могут возникнуть).